

ASSIGNMENT-12.4

Name: G.Mythili

Ht.no: 2303A51283

Batch: 05

Task 1: Bubble Sort for Ranking Exam Scores

Scenario

You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.

Task Description

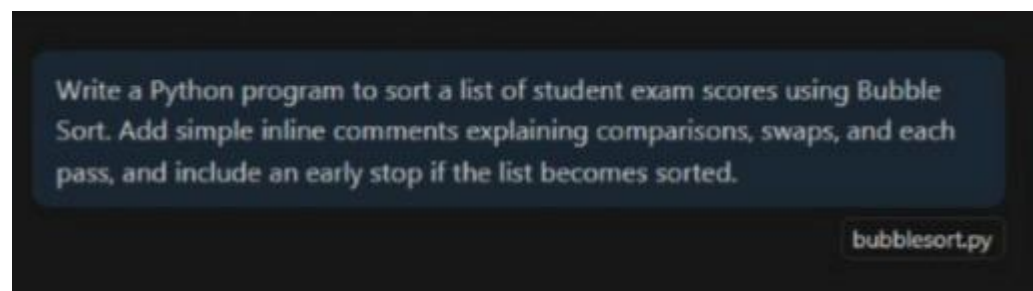
- Implement Bubble Sort in Python to sort a list of student scores.
- Use an AI tool to:

Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes Identify early termination

conditions when the list becomes sorted
Provide a brief time complexity analysis.

Prompt:

Write a Python program to sort a list of student exam scores using Bubble Sort. Add simple inline comments explaining comparisons, swaps, and each pass, and include an early stop if the list becomes sorted.



Code:

```

bubblesort.py > bubblesort
def bubble_sort(scores):
    """
    Sort a list of exam scores using Bubble Sort with early stopping.

    Args:
        scores: List of integers representing exam scores

    Returns:
        Sorted list in ascending order
    """
    n = len(scores)

    # Iterate through each pass
    for i in range(n):
        # Flag to detect if any swaps occur in this pass
        swapped = False

        # Compare adjacent elements
        for j in range(0, n - i - 1):
            # Compare current score with next score
            if scores[j] > scores[j + 1]:
                # Swap if current score is greater
                scores[j], scores[j + 1] = scores[j + 1], scores[j]
                swapped = True

        # Early stop: If no swaps occurred, list is already sorted
        if not swapped:
            break

    return scores

# Example usage
if __name__ == "__main__":
    exam_scores = [85, 92, 78, 95, 82, 88, 76, 90]

    print("Original scores:", exam_scores)
    sorted_scores = bubble_sort(exam_scores)
    print("Sorted scores:", sorted_scores)

```

Output:

```

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bubblesort.py"
Original scores: [85, 92, 78, 95, 82, 88, 76, 90]
Sorted scores: [76, 78, 82, 85, 88, 90, 92, 95]
PS C:\AI Assistant Coding>

```

Explanation:

- Bubble Sort compares two adjacent scores in the list.
- If the left score is greater than the right score, they are swapped.
- This process repeats for all elements in one pass.
- After each pass, the largest score moves to the end of the list.
- Multiple passes are done until the list becomes sorted.

- If no swaps happen in a pass, the algorithm stops early because the list is already sorted.

Time Complexity:

- Best Case: $O(n)$ – when the list is already sorted.
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$ – when the list is in reverse order.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario

You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates.

Task Description

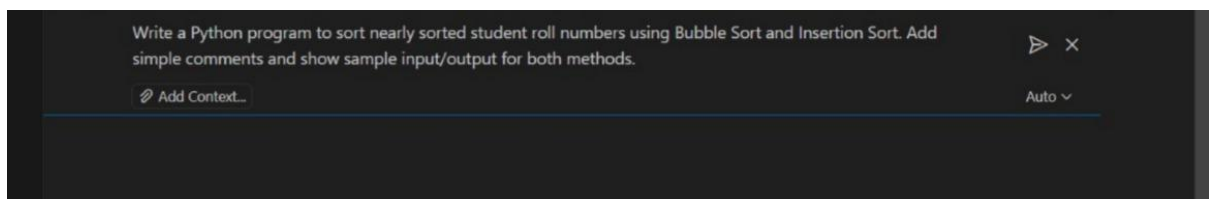
- Start with a Bubble Sort implementation.
- Ask AI to:

Review the problem and suggest a more suitable sorting algorithm
Generate an Insertion Sort implementation
Explain why Insertion Sort performs better on nearly sorted data.

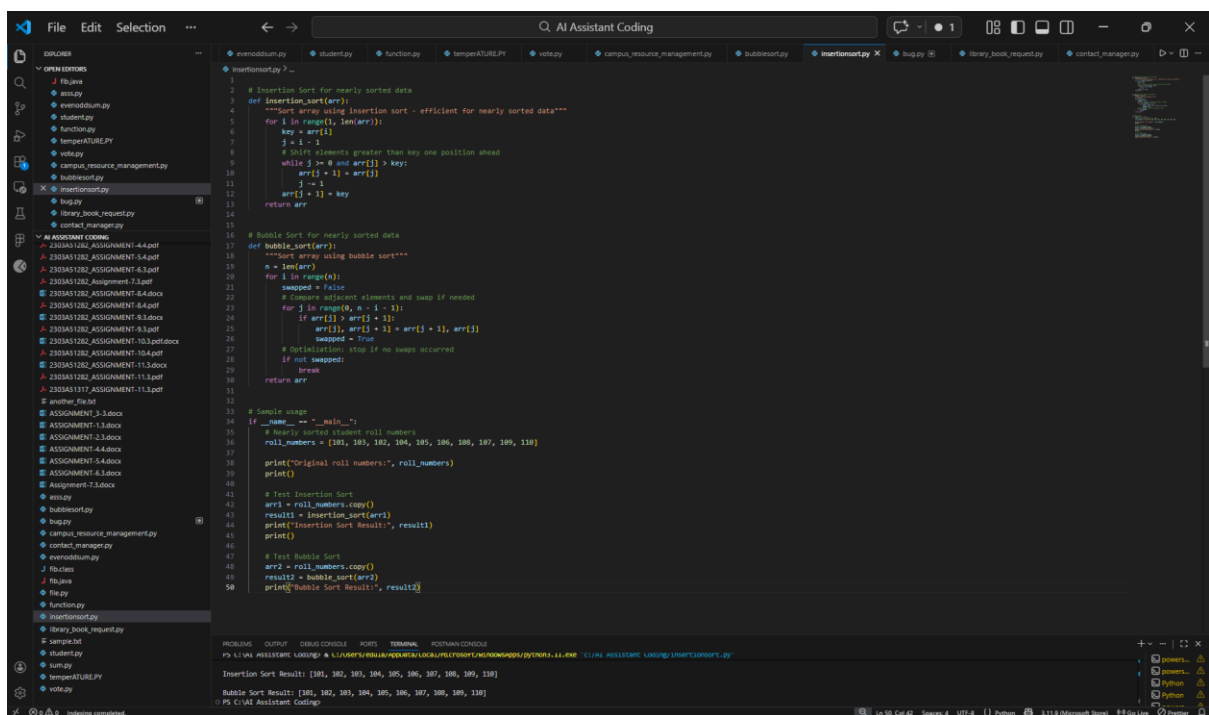
- Compare execution behavior on nearly sorted input

Prompt:

Write a Python program to sort nearly sorted student roll numbers using Bubble Sort and Insertion Sort. Add simple comments and show sample input/output for both methods.



Code:



Output:

```
PS C:\AI Assistant Coding > & C:/Users/edala/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bubblesort.py"
PS C:\AI Assistant Coding > & C:/Users/edala/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/insertionsort.py"
Original roll numbers: [101, 103, 102, 104, 105, 106, 100, 107, 109, 110]
Insertion Sort Result: [101, 102, 103, 104, 105, 106, 107, 100, 109, 110]
Bubble Sort Result: [101, 102, 103, 104, 105, 106, 107, 100, 109, 110]
PS C:\AI Assistant Coding >
```

Explanation:

The program sorts nearly sorted student roll numbers using two methods: Bubble Sort and Insertion Sort. Bubble Sort repeatedly compares and swaps adjacent elements until the list is sorted, while Insertion Sort places each element in its correct position by shifting larger elements to the right. In nearly sorted data, Insertion Sort is faster because only a few elements need shifting, so it completes in fewer operations compared to Bubble Sort.

Task 3: Searching Student Records in a Database Scenario

You are developing a student information portal where users search for student records by roll number.

Task Description

- Implement:

Linear Search for unsorted student data

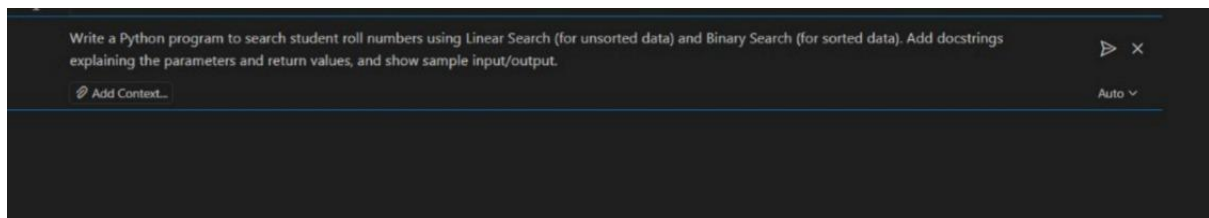
Binary Search for sorted student data

- Use AI to:

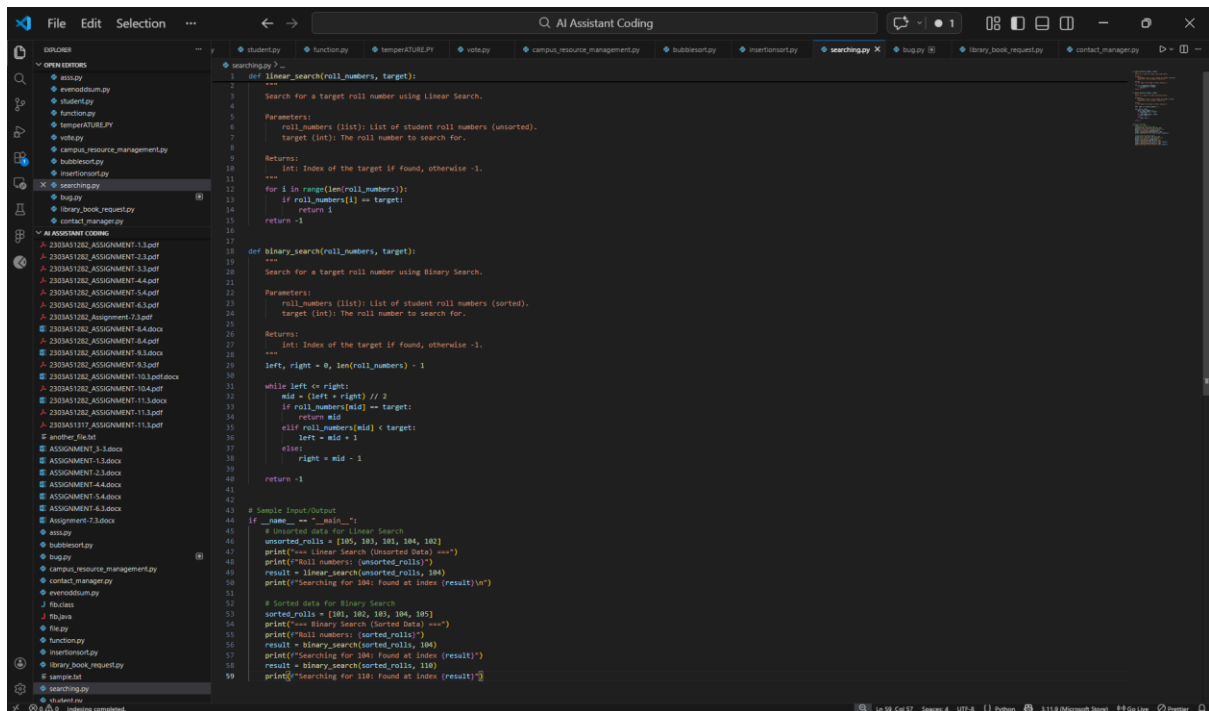
Add docstrings explaining parameters and return values ,Explain when Binary Search is applicable Highlight performance differences between the two searches

Prompt:

Write a Python program to search student roll numbers using Linear Search (for unsorted data) and Binary Search (for sorted data). Add docstrings explaining the parameters and return values, and show sample input/output.



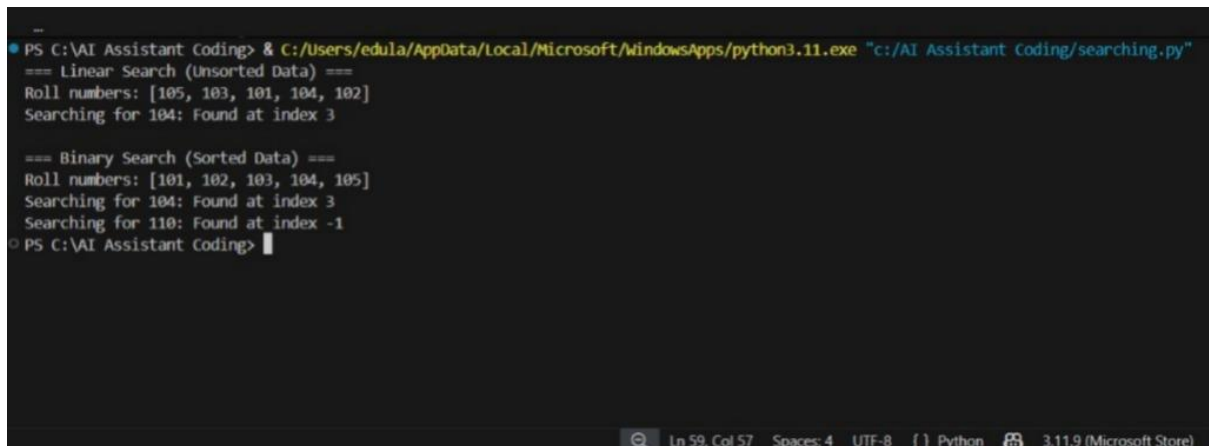
Code:



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer lists various files, including assignments and a file named 'searching.py'. The code editor displays the following Python code:

```
def linear_search(roll_numbers, target):  
    """  
    Search for a target roll number using Linear Search.  
    """  
    Parameters:  
    roll_numbers (list): List of student roll numbers (unsorted).  
    target (int): The roll number to search for.  
    Returns:  
    int: Index of the target if found, otherwise -1.  
    """  
    for i in range(len(roll_numbers)):  
        if roll_numbers[i] == target:  
            return i  
    return -1  
  
def binary_search(roll_numbers, target):  
    """  
    Search for a target roll number using Binary Search.  
    """  
    Parameters:  
    roll_numbers (list): List of student roll numbers (sorted).  
    target (int): The roll number to search for.  
    Returns:  
    int: Index of the target if found, otherwise -1.  
    """  
    left, right = 0, len(roll_numbers) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if roll_numbers[mid] == target:  
            return mid  
        elif roll_numbers[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1  
  
# Sample Input/Output  
if __name__ == "__main__":  
    # Unsorted data for Linear Search  
    unsorted_rolls = [105, 103, 101, 104, 102]  
    print("=== Linear Search (Unsorted Data) ===")  
    print("Roll numbers: (unsorted_rolls)")  
    result = linear_search(unsorted_rolls, 104)  
    print(f"Searching for 104: Found at index {result}")  
    # Sorted data for Binary Search  
    sorted_rolls = [101, 102, 103, 104, 105]  
    print("=== Binary Search (Sorted Data) ===")  
    print("Roll numbers: (sorted_rolls)")  
    result = binary_search(sorted_rolls, 104)  
    print(f"Searching for 104: Found at index {result}")  
    result = binary_search(sorted_rolls, 110)  
    print(f"Searching for 110: Found at index {result}")
```

Output:



The screenshot shows a terminal window with the following output:

```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/searching.py"  
=== Linear Search (Unsorted Data) ===  
Roll numbers: [105, 103, 101, 104, 102]  
Searching for 104: Found at index 3  
  
=== Binary Search (Sorted Data) ===  
Roll numbers: [101, 102, 103, 104, 105]  
Searching for 104: Found at index 3  
Searching for 110: Found at index -1  
PS C:\AI Assistant Coding>
```

Explanation:

- The program uses Linear Search to find a roll number in an unsorted list by checking each element one by one.
- It uses Binary Search for a sorted list by repeatedly dividing the list into halves until the roll number is found or not found.

- Binary Search is faster because it reduces the search space each step, while Linear Search checks all elements in the worst case.

Time Complexity:

- Linear Search: Best $O(1)$, Worst $O(n)$
- Binary Search: Best $O(1)$, Worst $O(\log n)$

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing Scenario

You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Task Description

- Provide AI with partially written recursive functions for:

Quick Sort

Merge Sort

- Ask AI to:

Complete the recursive logic

Add meaningful docstrings

Explain how recursion works in each algorithm

- Test both algorithms on:

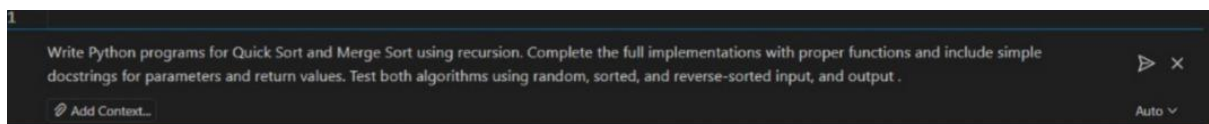
Random data

Sorted data

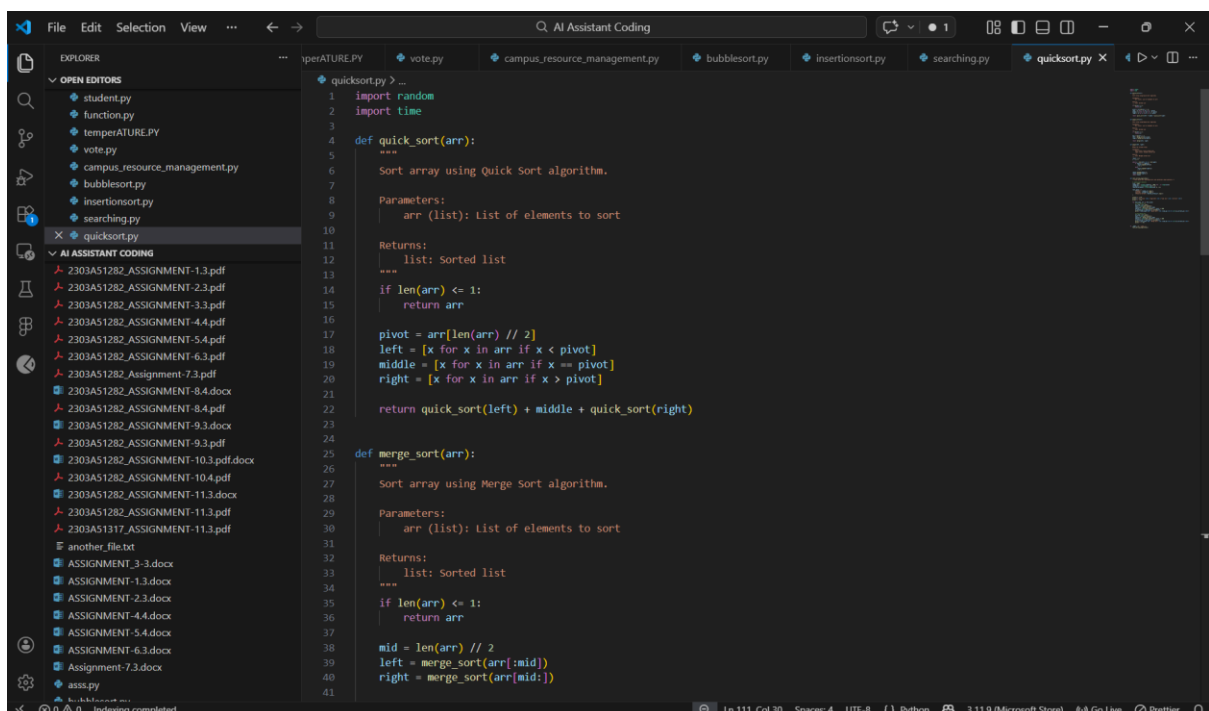
Reverse-sorted data

Prompt:

Write Python programs for Quick Sort and Merge Sort using recursion. Complete the full implementations with proper functions and include simple docstrings for parameters and return values. Test both algorithms using random, sorted, and reverse-sorted input, and output .



Code:



This screenshot shows the VS Code editor with the file explorer on the left and the code editor in the center. The file explorer shows a project structure with various Python and PDF files. The code editor displays the implementation of the merge sort algorithm in the file `quicksort.py`. The code includes a `merge_sort` function that recursively sorts an array by dividing it into two halves and then merging them back in sorted order. It also includes a `test_sorting_algorithms` function that generates test cases and prints the results.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        merge(left, right)
    return arr

def merge(left, right):
    """
    Merge two sorted arrays.

    Parameters:
        left (list): First sorted list
        right (list): Second sorted list

    Returns:
        list: Merged sorted list
    """
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def test_sorting_algorithms():
    """Test Quick Sort and Merge Sort with different input patterns."""
    # Create test datasets
    size = 1000
    random_arr = [random.randint(1, 1000) for _ in range(size)]
    sorted_arr = list(range(1, size + 1))
    reverse_sorted_arr = list(range(size, 0, -1))
    test_cases = [
```

This screenshot shows the VS Code editor with the file explorer on the left and the code editor in the center. The code editor displays the completion of the `test_sorting_algorithms` function in the file `quicksort.py`. The function now includes a loop to test both Quick Sort and Merge Sort on the generated test cases, printing the time taken and whether the result is correct. The `if __name__ == '__main__':` block is also present, calling the `test_sorting_algorithms` function.

```
test_cases = [
    ("Random", random_arr.copy()),
    ("Sorted", sorted_arr.copy()),
    ("Reverse Sorted", reverse_sorted_arr.copy())
]

print("\n" * 70)
print(f"{'Test Case':<20} {'Algorithm':<15} {'Time (ms)':<15} {'Correct':<10}")
print("\n" * 70)

for test_name, arr in test_cases:
    # Test Quick Sort
    arr_copy = arr.copy()
    start = time.perf_counter()
    result_qs = quick_sort(arr_copy)
    time_qs = (time.perf_counter() - start) * 1000
    is_correct_qs = result_qs == sorted(arr)
    print(f"{'Test Case':<20} {'Quick Sort':<15} {'Time (ms)':<15.4f} {'Correct':<10}")

    # Test Merge Sort
    arr_copy = arr.copy()
    start = time.perf_counter()
    result_ms = merge_sort(arr_copy)
    time_ms = (time.perf_counter() - start) * 1000
    is_correct_ms = result_ms == sorted(arr)
    print(f"{'Test Case':<20} {'Merge Sort':<15} {'Time (ms)':<15.4f} {'Correct':<10}")
    print("\n" * 70)

if __name__ == '__main__':
    test_sorting_algorithms()
```

Output:

```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe C:/AI Assistant Coding/quicksort.py
=====
Test Case      Algorithm      Time (ms)      Correct
-----
Random         Quick Sort     2.7398         True
Random         Merge Sort     2.5924         True
-----
Sorted         Quick Sort     1.9160         True
Sorted         Merge Sort     2.0663         True
-----
Reverse Sorted Quick Sort     1.9927         True
Reverse Sorted Merge Sort     2.5738         True
-----
PS C:\AI Assistant Coding>
```

Explanation:

- Both programs use recursion, meaning the function calls itself to sort smaller parts of the list.
- Quick Sort selects a pivot element and places smaller elements on the left and larger elements on the right, then recursively sorts both sides.
- Merge Sort divides the list into two halves, recursively sorts each half, and then merges the sorted halves together.

Time Complexity:

- Quick Sort: Best/Average $O(n \log n)$, Worst $O(n^2)$
- Merge Sort: Best/Average/Worst $O(n \log n)$

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.

Task Description

- Write a naive duplicate detection algorithm using nested loops.
- Use AI to:

Analyze the time complexity

Suggest an optimized approach using sets or dictionaries

Rewrite the algorithm with improved efficiency

- Compare execution behavior conceptually for large input sizes

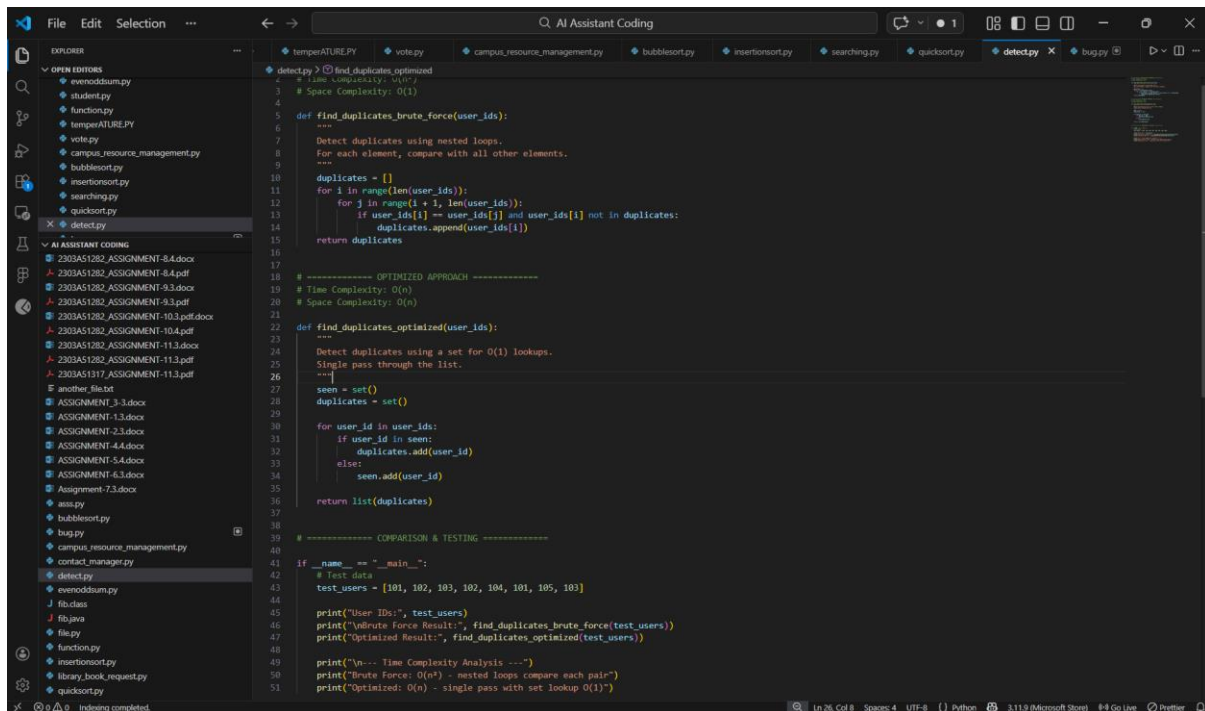
Prompt:

Write a Python program to detect duplicate user IDs using a nested loop (brute-force) method.

Analyze the time complexity and then suggest a faster approach using a set or dictionary. Rewrite the program using the optimized method and show both versions of the code for comparison.

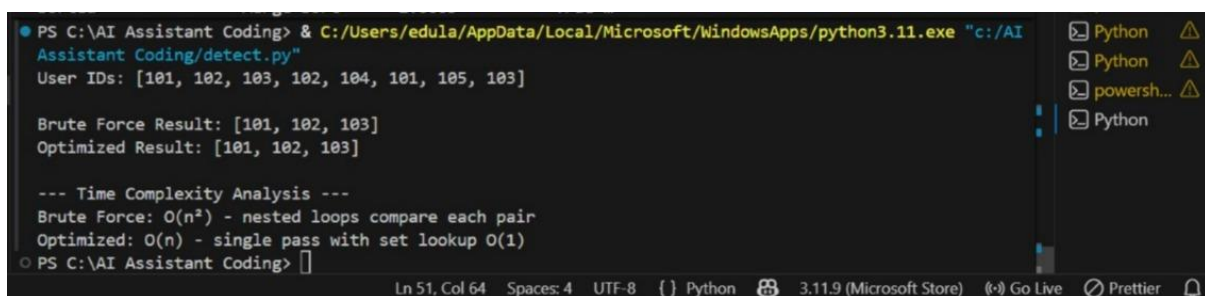
Write a Python program to detect duplicate user IDs using a nested loop (brute-force) method. Analyze the time complexity and then suggest a faster approach using a set or dictionary. Rewrite the program using the optimized method and show both versions of the code for comparison.

Code:



```
File Edit Selection ... Q AI Assistant Coding
temperture.py vote.py campus_resource_management.py bubblesort.py insertionsort.py searching.py quicksort.py detect.py bug.py
EXPLORER
  OPEN EDITORS
    evenoddsamp.py
    student.py
    function.py
    temperATURE.py
    vote.py
    campus_resource_management.py
    bubblesort.py
    insertionsort.py
    searching.py
    quicksort.py
    detect.py
  AI ASSISTANT CODING
    2303AS1282_ASSIGNMENT-8.4.docx
    2303AS1282_ASSIGNMENT-8.4.pdf
    2303AS1282_ASSIGNMENT-9.3.docx
    2303AS1282_ASSIGNMENT-9.3.pdf
    2303AS1282_ASSIGNMENT-10.3.pdf.docx
    2303AS1282_ASSIGNMENT-10.4.pdf
    2303AS1282_ASSIGNMENT-11.3.docx
    2303AS1282_ASSIGNMENT-11.3.pdf
    2303AS1317_ASSIGNMENT-11.3.pdf
    another_file.txt
    ASSIGNMENT_3-3.docx
    ASSIGNMENT-1.3.docx
    ASSIGNMENT-2.3.docx
    ASSIGNMENT-4.4.docx
    ASSIGNMENT-5.4.docx
    ASSIGNMENT-6.3.docx
    Assignment-7.3.docx
    ass.py
    bubblesort.py
    bug.py
    campus_resource_management.py
    contact_manager.py
    detect.py
    evenoddsamp.py
    fib.class
    fib.py
    fib.py
    function.py
    insertionsort.py
    library_book_request.py
    quicksort.py
  detect.py
    1 # Time Complexity: O(n^2)
    2 # Space Complexity: O(1)
    3
    4 def find_duplicates_brute_force(user_ids):
    5     """
    6     Detect duplicates using nested loops.
    7     For each element, compare with all other elements.
    8     """
    9     duplicates = []
    10     for i in range(len(user_ids)):
    11         for j in range(i + 1, len(user_ids)):
    12             if user_ids[i] == user_ids[j] and user_ids[i] not in duplicates:
    13                 duplicates.append(user_ids[i])
    14     return duplicates
    15
    16
    17 # ===== OPTIMIZED APPROACH =====
    18 # Time Complexity: O(n)
    19 # Space Complexity: O(n)
    20
    21 def find_duplicates_optimized(user_ids):
    22     """
    23     Detect duplicates using a set for O(1) lookups.
    24     Single pass through the list.
    25     """
    26     seen = set()
    27     duplicates = set()
    28     for user_id in user_ids:
    29         if user_id in seen:
    30             duplicates.add(user_id)
    31         else:
    32             seen.add(user_id)
    33     return list(duplicates)
    34
    35
    36 # ===== COMPARISON & TESTING =====
    37
    38 if __name__ == "__main__":
    39     # Test data
    40     test_users = [101, 102, 103, 102, 104, 101, 105, 103]
    41
    42     print("User IDs:", test_users)
    43     print("Brute Force Result:", find_duplicates_brute_force(test_users))
    44     print("Optimized Result:", find_duplicates_optimized(test_users))
    45
    46     print("\n--- Time Complexity Analysis ---")
    47     print("Brute Force: O(n^2) - nested loops compare each pair")
    48     print("Optimized: O(n) - single pass with set lookup O(1)")
```

Output:



```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/detect.py"
User IDs: [101, 102, 103, 102, 104, 101, 105, 103]

Brute Force Result: [101, 102, 103]
Optimized Result: [101, 102, 103]

--- Time Complexity Analysis ---
Brute Force: O(n^2) - nested loops compare each pair
Optimized: O(n) - single pass with set lookup O(1)
PS C:\AI Assistant Coding>
```

Explanation:

- In the brute-force method, the program checks each user ID with every other ID to find duplicates, so it takes more time.

- In the optimized method, the program uses a set to remember IDs and quickly detect if an ID appears again.
- The optimized method is faster because it scans the list only once, so it works better for large data.